

TikTok Project

Course 6 - The Nuts and bolts of machine learning

Recall that you are a data professional at TikTok. Your supervisor was impressed with the work you have done and has requested that you build a machine learning model that can be used to determine whether a video contains a claim or whether it offers an opinion. With a successful prediction model, TikTok can reduce the backlog of user reports and prioritize them more efficiently.

A notebook was structured and prepared to help you in this project. A notebook was structured and prepared to help you in this project. Please complete the following questions.

Course 6 End-of-course project: Classifying videos using machine learning

In this activity, you will practice using machine learning techniques to predict on a binary outcome variable.

The purpose of this model is to increase response time and system efficiency by automating the initial stages of the claims process.

The goal of this model is to predict whether a TikTok video presents a "claim" or presents an "opinion".

This activity has three parts:

Part 1: Ethical considerations

- Consider the ethical implications of the request
- Should the objective of the model be adjusted?

Part 2: Feature engineering

- Perform feature selection, extraction, and transformation to prepare the data for modeling

Part 3: Modeling

- Build the models, evaluate them, and advise on next steps

Follow the instructions and answer the questions below to complete the activity. Then, you will complete an Executive Summary using the questions listed on the PACE Strategy Document.

Be sure to complete this activity before moving on. The next course item will provide you with a completed exemplar to compare to your own work.

Classify videos using machine learning

PACE stages

Throughout these project notebooks, you'll see references to the problem-solving framework PACE. The following notebook components are labeled with the respective PACE stage: Plan, Analyze, Construct, and Execute.

PACE: Plan

Consider the questions in your PACE Strategy Document to reflect on the Plan stage.

In this stage, consider the following questions:

1. **What are you being asked to do? What metric should I use to evaluate success of my business/organizational objective?**
2. **What are the ethical implications of the model? What are the consequences of your model making errors?**
 - What is the likely effect of the model when it predicts a false negative (i.e., when the model says a video does not contain a claim and it actually does)?
 - What is the likely effect of the model when it predicts a false positive (i.e., when the model says a video does contain a claim and it actually does not)?
1. **How would you proceed?**

TikTok users can report videos that they believe violate the platform's terms of service. Because there are millions of TikTok videos created and viewed every day, this means that many videos get reported—too many to be individually reviewed by a human moderator.

Analysis indicates that when authors do violate the terms of service, they're much more likely to be presenting a claim than an opinion. Therefore, it is useful to be able to determine which videos make claims and which videos are opinions.

TikTok wants to build a machine learning model to help identify claims and opinions. Videos that are labeled opinions will be less likely to go on to be reviewed by a human moderator. Videos that are labeled as claims will be further sorted by a downstream process to determine whether they should get prioritized for review. For example, perhaps videos that are classified as claims would then be ranked by how many times they were reported, then the top x% would be reviewed by a human each day.

A machine learning model would greatly assist in the effort to present human moderators with videos that are most likely to be in violation of TikTok's terms of service.

Modeling workflow and model selection process

Previous work with this data has revealed that there are ~20,000 videos in the sample. This is sufficient to conduct a rigorous model validation workflow, broken into the following steps:

1. Split the data into train/validation/test sets (60/20/20)
2. Fit models and tune hyperparameters on the training set
3. Perform final model selection on the validation set
4. Assess the champion model's performance on the test set

Task 1. Imports and data loading

Start by importing packages needed to build machine learning models to achieve the goal of this project.

```
# Import packages for data manipulation
### YOUR CODE HERE ###
import pandas as pd
import numpy as np

# Import packages for data visualization
### YOUR CODE HERE ###
import matplotlib.pyplot as plt
import seaborn as sns

# Import packages for data preprocessing
### YOUR CODE HERE ###
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import classification_report, accuracy_score,
precision_score, \
    recall_score, f1_score, confusion_matrix,
ConfusionMatrixDisplay

# Import packages for data modeling
```

```

### YOUR CODE HERE ###
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from xgboost import plot_importance

```

Now load the data from the provided csv file into a dataframe.

Note: As shown in this cell, the dataset has been automatically loaded in for you. You do not need to download the .csv file, or provide more code, in order to access the dataset and proceed with this lab. Please continue with this activity by completing the following instructions.

```

# Load dataset into dataframe
data = pd.read_csv("tiktok_dataset.csv")

```

PACE: Analyze

Consider the questions in your PACE Strategy Document to reflect on the Analyze stage.

Task 2: Examine data, summary info, and descriptive stats

Inspect the first five rows of the dataframe.

```

# Display first few rows
### YOUR CODE HERE ###
data.head()

```

	#	claim_status	video_id	video_duration_sec	\
0	1	claim	7017666017	59	
1	2	claim	4014381136	32	
2	3	claim	9859838091	31	
3	4	claim	1866847991	25	
4	5	claim	7105231098	19	

	verified_status	\	video_transcription_text
0	someone shared with me that drone deliveries a...	not verified	
1	someone shared with me that there are more mic...	not verified	
2	someone shared with me that american industria...	not verified	
3	someone shared with me that the metro of st. p...	not verified	
4	someone shared with me that the number of busi...	not verified	

	author_ban_status	video_view_count	video_like_count	video_share_count	\
--	-------------------	------------------	------------------	-------------------	---

0	under review	343296.0	19425.0
241.0			
1	active	140877.0	77355.0
19034.0			
2	active	902185.0	97690.0
2858.0			
3	active	437506.0	239954.0
34812.0			
4	active	56167.0	34987.0
4110.0			

	video_download_count	video_comment_count
0	1.0	0.0
1	1161.0	684.0
2	833.0	329.0
3	1234.0	584.0
4	547.0	152.0

Get the number of rows and columns in the dataset.

```
# Get number of rows and columns
### YOUR CODE HERE ###
data.shape
(19382, 12)
```

Get the data types of the columns.

```
# Get data types of columns
### YOUR CODE HERE ###
data.dtypes

#
claim_status      int64
video_id          object
video_duration_sec int64
video_transcription_text object
verified_status   object
author_ban_status object
video_view_count  float64
video_like_count  float64
video_share_count float64
video_download_count float64
video_comment_count float64
dtype: object
```

Get basic information about the dataset.

```
# Get basic information
### YOUR CODE HERE ###
data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19382 entries, 0 to 19381
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   #                                       19382 non-null  int64
1   claim_status                          19084 non-null  object
2   video_id                              19382 non-null  int64
3   video_duration_sec                   19382 non-null  int64
4   video_transcription_text              19084 non-null  object
5   verified_status                      19382 non-null  object
6   author_ban_status                    19382 non-null  object
7   video_view_count                     19084 non-null  float64
8   video_like_count                     19084 non-null  float64
9   video_share_count                    19084 non-null  float64
10  video_download_count                  19084 non-null  float64
11  video_comment_count                   19084 non-null  float64
dtypes: float64(5), int64(3), object(4)
memory usage: 1.8+ MB
```

Generate basic descriptive statistics about the dataset.

```
# Generate basic descriptive stats
### YOUR CODE HERE ###
data.describe().T
```

	count	mean	std
min \			
#	19382.0	9.691500e+03	5.595246e+03
1.000000e+00			
video_id	19382.0	5.627454e+09	2.536440e+09
1.234959e+09			
video_duration_sec	19382.0	3.242173e+01	1.622997e+01
5.000000e+00			
video_view_count	19084.0	2.547086e+05	3.228933e+05
2.000000e+01			
video_like_count	19084.0	8.430464e+04	1.334205e+05
0.000000e+00			
video_share_count	19084.0	1.673525e+04	3.203617e+04
0.000000e+00			
video_download_count	19084.0	1.049430e+03	2.004300e+03
0.000000e+00			
video_comment_count	19084.0	3.493121e+02	7.996389e+02
0.000000e+00			

	25%	50%	75%
max			
#	4.846250e+03	9.691500e+03	1.453675e+04
1.938200e+04			
video_id	3.430417e+09	5.618664e+09	7.843960e+09
9.999873e+09			
video_duration_sec	1.800000e+01	3.200000e+01	4.700000e+01
6.000000e+01			
video_view_count	4.942500e+03	9.954500e+03	5.043270e+05
9.998170e+05			
video_like_count	8.107500e+02	3.403500e+03	1.250200e+05
6.578300e+05			
video_share_count	1.150000e+02	7.170000e+02	1.822200e+04
2.561300e+05			
video_download_count	7.000000e+00	4.600000e+01	1.156250e+03
1.499400e+04			
video_comment_count	1.000000e+00	9.000000e+00	2.920000e+02
9.599000e+03			

Check for and handle missing values.

```
# Check for missing values
### YOUR CODE HERE ###
data.isna().sum()

#
#          0
claim_status      298
video_id          0
video_duration_sec 0
video_transcription_text 298
verified_status    0
author_ban_status  0
video_view_count   298
video_like_count    298
video_share_count   298
video_download_count 298
video_comment_count 298
dtype: int64

# Drop rows with missing values
### YOUR CODE HERE ###
data = data.dropna(axis=0)

# Display first few rows after handling missing values
### YOUR CODE HERE ###
display(data.head())
display(data.isna().sum())
```

#	claim_status	video_id	video_duration_sec	\
0	1	claim 7017666017	59	
1	2	claim 4014381136	32	
2	3	claim 9859838091	31	
3	4	claim 1866847991	25	
4	5	claim 7105231098	19	

	video_transcription_text	verified_status	\
0	someone shared with me that drone deliveries a...	not verified	
1	someone shared with me that there are more mic...	not verified	
2	someone shared with me that american industria...	not verified	
3	someone shared with me that the metro of st. p...	not verified	
4	someone shared with me that the number of busi...	not verified	

	author_ban_status	video_view_count	video_like_count	video_share_count	\
0	under review	343296.0	19425.0	241.0	
1	active	140877.0	77355.0	19034.0	
2	active	902185.0	97690.0	2858.0	
3	active	437506.0	239954.0	34812.0	
4	active	56167.0	34987.0	4110.0	

	video_download_count	video_comment_count
0	1.0	0.0
1	1161.0	684.0
2	833.0	329.0
3	1234.0	584.0
4	547.0	152.0

#	0
claim_status	0
video_id	0
video_duration_sec	0
video_transcription_text	0
verified_status	0
author_ban_status	0
video_view_count	0
video_like_count	0
video_share_count	0

```
video_download_count    0
video_comment_count     0
dtype: int64
```

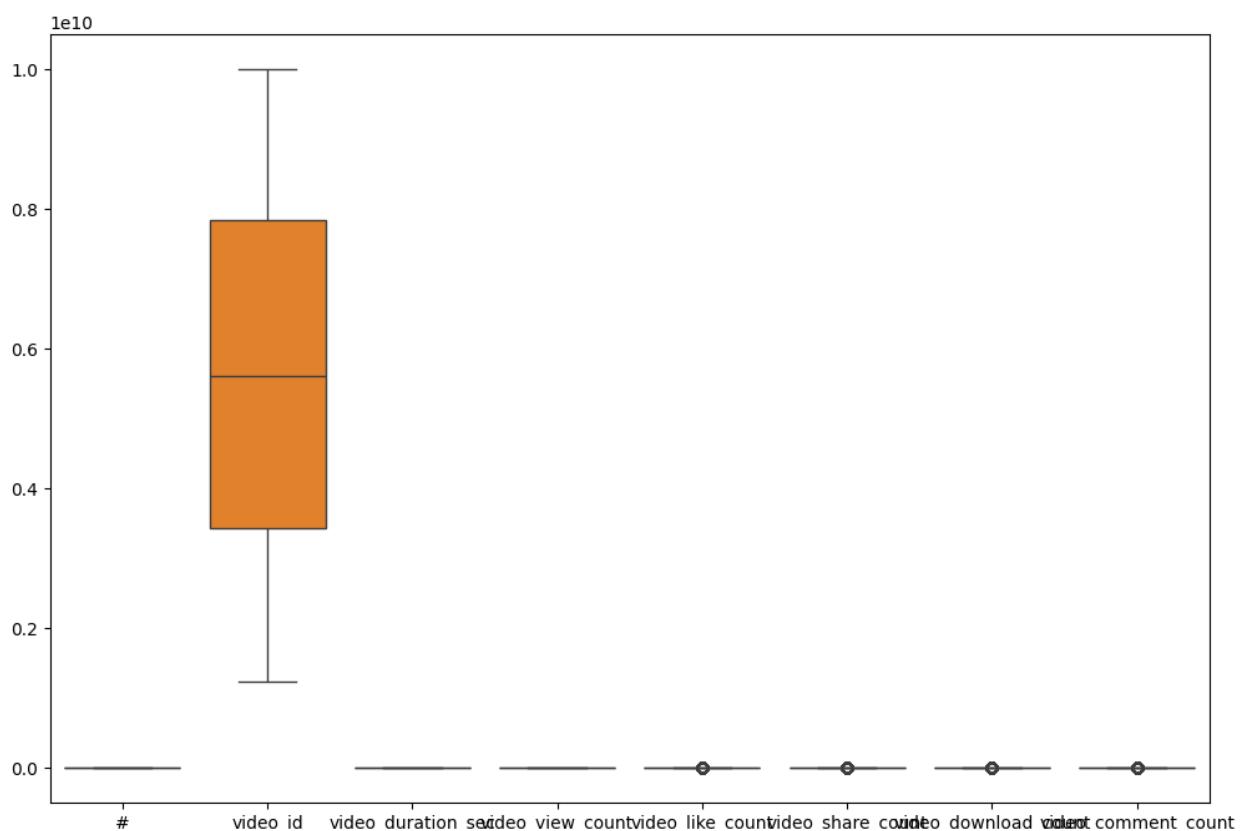
Check for and handle duplicates.

```
# Check for duplicates
### YOUR CODE HERE ###
data.duplicated().sum()

0
```

Check for and handle outliers.

```
### YOUR CODE HERE ###
plt.subplots(figsize=(12,8))
sns.boxplot(data)
plt.show()
```



Tree-based models are robust to outliers, so there is no need to impute or drop any values based on where they fall in their distribution.

Check class balance.

```
# Check class balance
### YOUR CODE HERE ###
data['claim_status'].value_counts(normalize=True)

claim_status
claim      0.503458
opinion    0.496542
Name: proportion, dtype: float64
```

Approximately 50.3% of the dataset represents claims and 49.7% represents opinions, so the outcome variable is balanced.

PACE: Construct

Consider the questions in your PACE Strategy Document to reflect on the Construct stage.

Task 3: Feature engineering

Extract the length of each `video_transcription_text` and add this as a column to the dataframe, so that it can be used as a potential feature in the model.

```
# Extract the length of each `video_transcription_text` and add this
as a column to the dataframe
### YOUR CODE HERE ###
data['text_length'] = data['video_transcription_text'].str.len()
data.head()
```

	#	claim_status	video_id	video_duration_sec	\
0	1	claim	7017666017	59	
1	2	claim	4014381136	32	
2	3	claim	9859838091	31	
3	4	claim	1866847991	25	
4	5	claim	7105231098	19	

	verified_status	\	video_transcription_text
0	someone shared with me that drone deliveries a...	not verified	
1	someone shared with me that there are more mic...	not verified	
2	someone shared with me that american industria...	not verified	
3	someone shared with me that the metro of st. p...	not verified	
4	someone shared with me that the number of busi...	not verified	

author_ban_status	video_view_count	video_like_count
-------------------	------------------	------------------

video_share_count \			
0	under review	343296.0	19425.0
241.0			
1	active	140877.0	77355.0
19034.0			
2	active	902185.0	97690.0
2858.0			
3	active	437506.0	239954.0
34812.0			
4	active	56167.0	34987.0
4110.0			

	video_download_count	video_comment_count	text_length
0	1.0	0.0	97
1	1161.0	684.0	107
2	833.0	329.0	137
3	1234.0	584.0	131
4	547.0	152.0	128

Calculate the average text_length for claims and opinions.

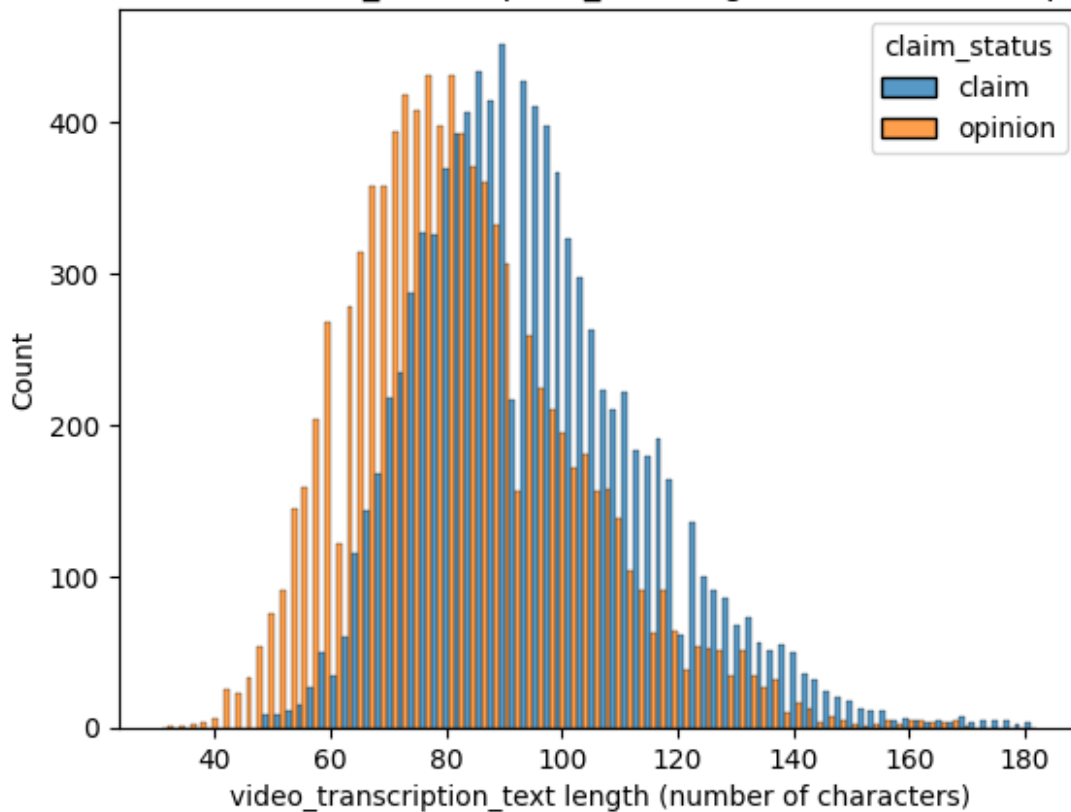
```
# Calculate the average text_length for claims and opinions
### YOUR CODE HERE ###
data.groupby("claim_status")[['text_length']].mean()
```

	text_length
claim_status	
claim	95.376978
opinion	82.722562

Visualize the distribution of text_length for claims and opinions.

```
# Visualize the distribution of `text_length` for claims and opinions
# Create two histograms in one plot
### YOUR CODE HERE ###
sns.histplot(
    data=data,
    x='text_length',
    stat='count',
    multiple='dodge',
    hue='claim_status'
)
plt.xlabel("video_transcription_text length (number of characters)")
plt.ylabel("Count")
plt.title("Distribution of video_transcription_text length for claims
and opinions")
plt.show()
```

Distribution of video_transcription_text length for claims and opinions



Feature selection and transformation

Encode target and categorical variables.

```
# Create a copy of the X data
### YOUR CODE HERE ###
X = data.copy()

# Drop unnecessary columns
### YOUR CODE HERE ###
X = X.drop(['#', 'video_id', 'claim_status'], axis=1)

# Encode target variable
### YOUR CODE HERE ###
y = data['claim_status'].replace({'opinion': 0, 'claim': 1})

# Dummy encode remaining categorical values
### YOUR CODE HERE ###
X = pd.get_dummies(X, columns=['verified_status',
                              'author_ban_status'], drop_first=True)

display(X.head())
display(y.head())
```

	video_duration_sec	video_transcription_text \
0	59	someone shared with me that drone deliveries a...
1	32	someone shared with me that there are more mic...
2	31	someone shared with me that american industria...
3	25	someone shared with me that the metro of st. p...
4	19	someone shared with me that the number of busi...

	video_view_count	video_like_count	video_share_count \
0	343296.0	19425.0	241.0
1	140877.0	77355.0	19034.0
2	902185.0	97690.0	2858.0
3	437506.0	239954.0	34812.0
4	56167.0	34987.0	4110.0

	video_download_count	video_comment_count	text_length \
0	1.0	0.0	97
1	1161.0	684.0	107
2	833.0	329.0	137
3	1234.0	584.0	131
4	547.0	152.0	128

	verified_status_verified	author_ban_status_banned \
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False

	author_ban_status_under review
0	True
1	False
2	False
3	False
4	False

0	1
1	1
2	1
3	1
4	1

Name: claim_status, dtype: int64

Task 4: Split the data

Assign target variable.

```
# Isolate target variable
### YOUR CODE HERE ###

# already done
```

Isolate the features.

```
# Isolate features
### YOUR CODE HERE ###

# Display first few rows of features dataframe
### YOUR CODE HERE ###

# already done
```

Task 5: Create train/validate/test sets

Split data into training and testing sets, 80/20.

```
# Split the data into training and testing sets
### YOUR CODE HERE ###
X_tr, X_test, y_tr, y_test = train_test_split(X, y, test_size=0.2,
random_state=0)
```

Split the training set into training and validation sets, 75/25, to result in a final ratio of 60/20/20 for train/validate/test sets.

```
# Split the training data into training and validation sets
### YOUR CODE HERE ###
X_train, X_val, y_train, y_val = train_test_split(X_tr, y_tr,
test_size=0.25, random_state=0)
```

Confirm that the dimensions of the training, validation, and testing sets are in alignment.

```
# Get shape of each training, validation, and testing set
### YOUR CODE HERE ###
X_train.shape, X_val.shape, X_test.shape, y_train.shape, y_val.shape,
y_test.shape

((11450, 11), (3817, 11), (3817, 11), (11450,), (3817,), (3817,))
```

The number of features (11) aligns between the training and testing sets.

- The number of rows aligns between the features and the outcome variable for training (11,450) and both validation and testing data (3,817).

BONUS CONTENT - Tokenize text column

NOTE: You are not expected to do this or know this, but you might find it useful and/or interesting to understand some basic ideas behind natural language processing (NLP), because of the nature of the data provided in this TikTok project.

The feature `video_transcription_text` is text-based. It is not a categorical variable, since it does not have a fixed number of possible values. One way to extract numerical features from it is through a bag-of-words algorithm like `CountVectorizer`.

`CountVectorizer` works by splitting text into n-grams, which are groups of n consecutive words. For instance, "a dime for a cup of coffee" (phrase A) broken into 2-grams would result in six two-word combinations:

a dime | dime for | for a | a cup | cup of | of coffee |

Then, the next sample's text would be parsed into 2-grams. So, "ask for a cup for a child" (phrase B) would result in:

ask for | for a | a cup | cup for | for a | a child |

This process would repeat for each observation in the dataset, and each n-gram would be treated like a distinct feature. Then, the text of each observation is compared to the full array of n-grams, and the numbers of occurrences are tallied:

	a dime	dime for	for a	a cup	cup of	of coffe e	ask for	cup for	a child
phrase A	1	1	1	1	1	1	0	0	0
phrase B	0	0	2	1	1	0	1	1	1
TOTAL	1	1	3	2	2	1	1	1	1

This would happen for the text of each observation in the data, and the text of each observation is parsed to get tallies for all the 2-word phrases from the entire data set for each observation, creating a large matrix.

If text is broken into 1-grams, then each feature in the matrix is an individual word.

After the count matrix has been created, `CountVectorizer` lets you choose to keep only the most frequently occurring n-grams. You specify how many. The n-grams that you select can then be used as features in a model.

Splitting text into n-grams is an example of tokenization. Tokenization is the process of breaking text into smaller units to derive meaning from the resulting tokens.

This notebook breaks each video's transcription text into both 2-grams and 3-grams, then takes the 15 most frequently occurring tokens from the entire dataset to use as features.

```

# Set up a `CountVectorizer` object, which converts a collection of
text to a matrix of token counts
from sklearn.feature_extraction.text import CountVectorizer

count_vec = CountVectorizer(ngram_range=(2, 3),
                             max_features=15,
                             stop_words='english')

count_vec

CountVectorizer(max_features=15, ngram_range=(2, 3),
stop_words='english')

```

Fit the vectorizer to the training data (generate the n-grams) and transform it (tally the occurrences). Only fit to the training data, not the validation or test data.

```

count_data =
count_vec.fit_transform(X_train['video_transcription_text']).toarray()
count_data

array([[0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       ...,
       [0, 0, 1, ..., 1, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0]])

# Place the numerical representation of `video_transcription_text`
from training set into a dataframe
count_df = pd.DataFrame(data=count_data,
columns=count_vec.get_feature_names_out())

# Display first few rows
count_df.head()

```

	colleague discovered	colleague learned	colleague read	discovered
news \				
0	0	0	0	
0				
1	0	0	0	
0				
2	0	0	0	
0				
3	0	0	0	
0				
4	0	0	0	
0				
	discussion board	friend learned	friend read	internet forum \
0	0	0	0	0

1	0	0	1	0
2	0	0	1	0
3	0	0	0	0
4	0	0	0	0

	learned media	media claim	news claim	point view	read media	\
0	0	0	0	0	0	
1	0	0	0	0	0	
2	0	0	0	0	0	
3	0	0	0	0	0	
4	0	0	0	0	0	

	social media	willing wager
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0

Concatenate `X\_train` and `count\_df` to form the final dataframe for training data (`X\_train\_final`)

Note: Using `.reset\_index(drop=True)` to reset the index in X_train after dropping `video\_transcription\_text`,

so that the indices align with those in `X\_train` and `count\_df`

X_train_final =

pd.concat([X_train.drop(columns=['video_transcription_text']).reset_index(drop=True), count_df], axis=1)

Display first few rows

X_train_final.head()

	video_duration_sec	video_view_count	video_like_count
video_share_count \			
0	51	2487.0	310.0
20.0			
1	43	118512.0	3543.0
374.0			
2	22	105902.0	1885.0
229.0			
3	17	9245.0	1670.0
440.0			
4	18	3791.0	660.0
63.0			

	video_download_count	video_comment_count	text_length	\
0	1.0	0.0	134	
1	70.0	29.0	97	
2	39.0	9.0	76	
3	13.0	4.0	58	
4	9.0	1.0	57	

	verified_status_verified	author_ban_status_banned	\
0	False	False	
1	False	True	
2	False	False	
3	False	False	
4	False	False	

	author_ban_status_under review	...	friend learned	friend read	\
0	False	...	0	0	
1	False	...	0	1	
2	True	...	0	1	
3	False	...	0	0	
4	False	...	0	0	

	internet forum	learned media	media claim	news claim	point view
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0

	read media	social media	willing wager
0	0	0	0
1	0	0	0
2	0	0	0
3	0	0	0
4	0	0	0

[5 rows x 25 columns]

Get n-gram counts for the validation data. Notice that the vectorizer is not being refit to the validation data. It's only transforming it. In other words, the transcriptions of the videos in the validation data are only being checked against the n-grams found in the training data.

```
# Extract numerical features from `video_transcription_text` in the
testing set
validation_count_data =
count_vec.transform(X_val['video_transcription_text']).toarray()
validation_count_data

array([[0, 0, 0, ..., 1, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 1, 0, 0],
```

```

...,
[0, 0, 0, ..., 0, 0, 0],
[0, 1, 0, ..., 0, 0, 0],
[0, 0, 0, ..., 0, 0, 0]])

```

Place the numerical representation of `video\_transcription\_text` from validation set into a dataframe

```

validation_count_df = pd.DataFrame(data=validation_count_data,
columns=count_vec.get_feature_names_out())
validation_count_df.head()

```

	colleague discovered	colleague learned	colleague read	discovered
news \				
0	0	0	0	
0				
1	0	0	0	
0				
2	0	0	0	
0				
3	0	0	0	
0				
4	0	0	1	
0				

	discussion board	friend learned	friend read	internet forum	\
0	0	0	0	0	
1	0	0	0	0	
2	0	0	1	0	
3	0	0	0	0	
4	0	0	0	0	

	learned media	media claim	news claim	point view	read media	\
0	0	0	0	0	1	
1	0	0	0	0	0	
2	0	0	0	0	1	
3	0	0	0	0	0	
4	0	0	0	0	1	

	social media	willing wager
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0

Concatenate `X\_val` and `validation\_count\_df` to form the final dataframe for training data (`X\_val\_final`)

Note: Using `.reset\_index(drop=True)` to reset the index in X_val after dropping `video\_transcription\_text`,

so that the indices align with those in `validation\_count\_df`

```
X_val_final =
pd.concat([X_val.drop(columns=['video_transcription_text']).reset_index(drop=True), validation_count_df], axis=1)
```

```
# Display first few rows
```

```
X_val_final.head()
```

	video_duration_sec	video_view_count	video_like_count
0	11	578891.0	379596.0
1	24	6255.0	1709.0
2	35	410356.0	249714.0
3	58	406911.0	25986.0
4	47	972573.0	138167.0

	video_download_count	video_comment_count	text_length
0	6591.0	44.0	72
1	13.0	1.0	96
2	2060.0	1252.0	88
3	564.0	248.0	83
4	3331.0	968.0	104

	verified_status_verified	author_ban_status_banned
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False

	author_ban_status_under review	...	friend learned	friend read
0	True	...	0	0
1	False	...	0	0
2	False	...	0	1
3	False	...	0	0
4	True	...	0	0

	internet forum	learned media	media claim	news claim	point view
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0

4	0	0	0	0	0
---	---	---	---	---	---

	read media	social media	willing wager
0	1	0	0
1	0	0	0
2	1	0	0
3	0	0	0
4	1	0	0

[5 rows x 25 columns]

Repeat the process to get n-gram counts for the test data. Again, don't refit the vectorizer to the test data. Just transform it.

```
# Extract numerical features from `video_transcription_text` in the
testing set
test_count_data =
count_vec.transform(X_test['video_transcription_text']).toarray()

# Place the numerical representation of `video_transcription_text`
from test set into a dataframe
test_count_df = pd.DataFrame(data=test_count_data,
columns=count_vec.get_feature_names_out())

# Concatenate `X_val` and `validation_count_df` to form the final
dataframe for training data (`X_val_final`)
X_test_final =
pd.concat([X_test.drop(columns=['video_transcription_text']
).reset_index(drop=True),
test_count_df], axis=1)
X_test_final.head()
```

	video_duration_sec	video_view_count	video_like_count
0	54	692084.0	135956.0
1	37	5164.0	1858.0
2	39	801951.0	344163.0
3	44	6429.0	2314.0
4	26	555780.0	106863.0

	video_download_count	video_comment_count	text_length
0	622.0	312.0	107
1	17.0	1.0	69

2	8578.0	2942.0	90
3	17.0	5.0	76
4	252.0	91.0	72

	verified_status_verified	author_ban_status_banned	\
0	False	False	
1	True	False	
2	False	False	
3	False	False	
4	False	False	

	author_ban_status_under review	...	friend learned	friend read	\
0	False	...	0	0	
1	True	...	0	0	
2	False	...	0	0	
3	False	...	0	0	
4	False	...	0	0	

	internet forum	learned media	media claim	news claim	point view
0	0	1	1	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0

	read media	social media	willing wager
0	0	0	0
1	0	0	0
2	0	0	0
3	0	0	0
4	0	0	0

[5 rows x 25 columns]

Task 6. Build models

Build a random forest model

Fit a random forest model to the training set. Use cross-validation to tune the hyperparameters and select the model that performs best on recall.

```
# Instantiate the random forest classifier
### YOUR CODE HERE ###
rf = RandomForestClassifier(random_state=0)
```

```

# Create a dictionary of hyperparameters to tune
### YOUR CODE HERE ###
cv_params = {'max_depth': [5, 7, None],
             'max_features': [0.3, 0.6],
             # 'max_features': 'auto'
             'max_samples': [0.7],
             'min_samples_leaf': [1, 2],
             'min_samples_split': [2, 3],
             'n_estimators': [75, 100, 200],
             }

# Define a list of scoring metrics to capture
### YOUR CODE HERE ###
scoring = ['accuracy', 'precision', 'recall', 'f1']

# Instantiate the GridSearchCV object
### YOUR CODE HERE ###
rf_cv = GridSearchCV(rf, cv_params, scoring=scoring, cv=5,
                    refit='recall')

%%time
### Fit the model to the data
### YOUR CODE HERE ###

rf_cv.fit(X_train_final, y_train)

CPU times: user 7min 44s, sys: 943 ms, total: 7min 45s
Wall time: 7min 45s

GridSearchCV(cv=5, estimator=RandomForestClassifier(random_state=0),
             param_grid={'max_depth': [5, 7, None], 'max_features':
[0.3, 0.6],
                        'max_samples': [0.7], 'min_samples_leaf': [1,
2],
                        'min_samples_split': [2, 3],
                        'n_estimators': [75, 100, 200]}},
             refit='recall', scoring=['accuracy', 'precision',
'recall', 'f1'])

# Examine best recall score
### YOUR CODE HERE ###
rf_cv.best_score_

0.9948228253467271

# Examine best parameters
### YOUR CODE HERE ###
rf_cv.best_params_

```

```
{'max_depth': None,
 'max_features': 0.6,
 'max_samples': 0.7,
 'min_samples_leaf': 1,
 'min_samples_split': 2,
 'n_estimators': 200}
```

Check the precision score to make sure the model isn't labeling everything as claims. You can do this by using the `cv_results_` attribute of the fit `GridSearchCV` object, which returns a numpy array that can be converted to a pandas dataframe. Then, examine the `mean_test_precision` column of this dataframe at the index containing the results from the best model. This index can be accessed by using the `best_index_` attribute of the fit `GridSearchCV` object.

```
# Access the GridSearch results and convert it to a pandas df
### YOUR CODE HERE ###
rf_results_df = pd.DataFrame(rf_cv.cv_results_)

# Examine the GridSearch results df at column 'mean_test_precision' in
# the best index
### YOUR CODE HERE ###
rf_results_df['mean_test_precision'][rf_cv.best_index_]

0.9994806638131711
```

Question: How well is your model performing? Consider average recall score and precision score.

Build an XGBoost model

```
# Instantiate the XGBoost classifier
### YOUR CODE HERE ###
xgb = XGBClassifier(objective='binary:logistic', random_state=0)

# Create a dictionary of hyperparameters to tune
### YOUR CODE HERE ###
cv_params = {'max_depth': [4, 8, 12],
             'min_child_weight': [3, 5],
             'learning_rate': [0.01, 0.1],
             'n_estimators': [300, 500]
            }

# Define a list of scoring metrics to capture
### YOUR CODE HERE ###
scoring = ['accuracy', 'precision', 'recall', 'f1']

# Instantiate the GridSearchCV object
### YOUR CODE HERE ###
```

```

xgb_cv = GridSearchCV(xgb, cv_params, scoring=scoring, cv=5,
refit='recall')

%%time
# Fit the model to the data
### YOUR CODE HERE ###

xgb_cv.fit(X_train_final, y_train)

CPU times: user 1min 31s, sys: 1.43 s, total: 1min 33s
Wall time: 47.4 s

GridSearchCV(cv=5,
             estimator=XGBClassifier(base_score=None, booster=None,
                                     callbacks=None,
colsample_bylevel=None,
                                     colsample_bynode=None,
                                     colsample_bytree=None,
device=None,
                                     early_stopping_rounds=None,
                                     enable_categorical=False,
eval_metric=None,
                                     feature_types=None, gamma=None,
                                     grow_policy=None,
importance_type=None,
                                     interaction_constraints=None,
                                     learning_rate=None, ...
                                     max_delta_step=None,
max_depth=None,
                                     max_leaves=None,
min_child_weight=None,
                                     missing=nan,
monotone_constraints=None,
                                     multi_strategy=None,
n_estimators=None,
                                     n_jobs=None,
num_parallel_tree=None,
                                     random_state=0, ...),
             param_grid={'learning_rate': [0.01, 0.1], 'max_depth':
[4, 8, 12],
                         'min_child_weight': [3, 5],
                         'n_estimators': [300, 500]},
             refit='recall', scoring=['accuracy', 'precision',
'recall', 'f1'])

# Examine best recall score
### YOUR CODE HERE ###
xgb_cv.best_score_

0.9898176171763818

```

```
# Examine best parameters
### YOUR CODE HERE ###
xgb_cv.best_params_

{'learning_rate': 0.1,
 'max_depth': 4,
 'min_child_weight': 5,
 'n_estimators': 300}
```

Repeat the steps used for random forest to examine the precision score of the best model identified in the grid search.

```
# Access the GridSearch results and convert it to a pandas df
xgb_results_df = pd.DataFrame(xgb_cv.cv_results_)

# Examine the GridSearch results df at column 'mean_test_precision' in
# the best index
xgb_results_df['mean_test_precision'][xgb_cv.best_index_]

0.9989540885869099
```

Question: How well does your model perform? Consider recall score and precision score.

This model also performs exceptionally well. Although both its precision and recall scores are very slightly lower than the random forest model's.

PACE: Execute

Consider the questions in your PACE Strategy Document to reflect on the Execute stage.

Task 7. Evaluate model

Evaluate models against validation criteria.

Random forest

```
# Use the random forest "best estimator" model to get predictions on
# the validation set
### YOUR CODE HERE ###
y_pred = rf_cv.best_estimator_.predict(X_val_final)
```

Display the predictions on the validation set.

```
# Display the predictions on the validation set
### YOUR CODE HERE ###
y_pred

array([1, 0, 1, ..., 1, 1, 1])
```

Display the true labels of the validation set.

```
# Display the true labels of the validation set
### YOUR CODE HERE ###
y_val

5846      1
12058     0
2975      1
8432      1
6863      1
      ..
6036      1
6544      1
2781      1
6426      1
4450      1
Name: claim_status, Length: 3817, dtype: int64
```

Create a confusion matrix to visualize the results of the classification model.

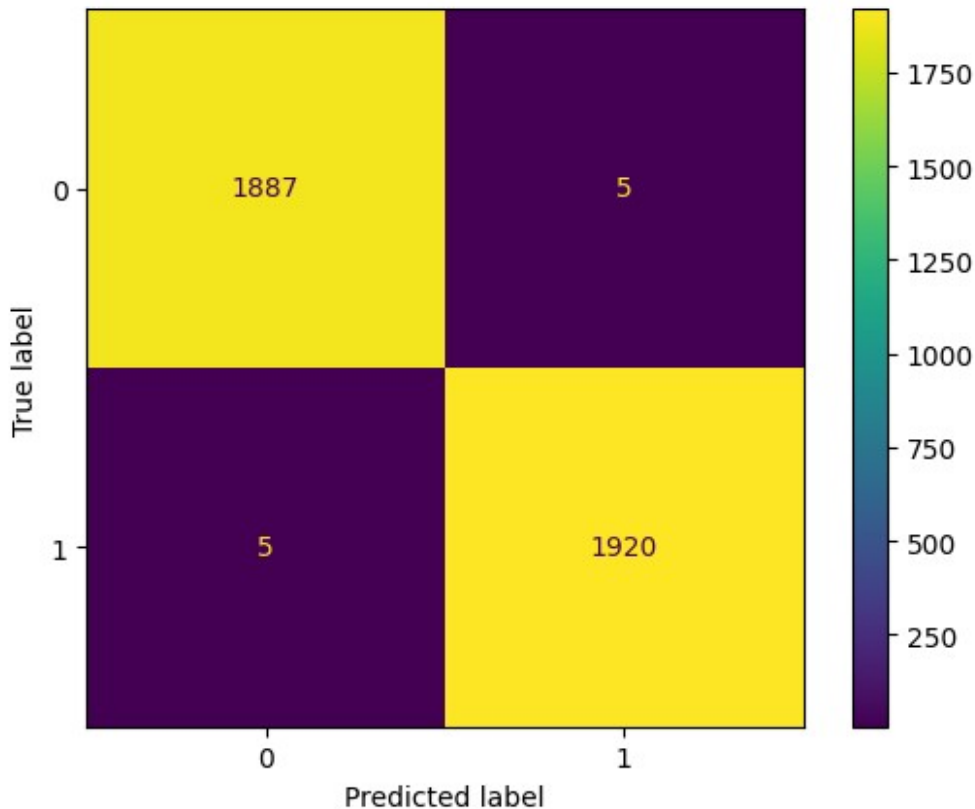
```
# Create a confusion matrix to visualize the results of the
classification model

# Compute values for confusion matrix
log_cm = confusion_matrix(y_val, y_pred)

# Create display of confusion matrix
log_disp = ConfusionMatrixDisplay(confusion_matrix=log_cm,
display_labels=None)

# Plot confusion matrix
log_disp.plot()

# Display plot
plt.show()
```



The upper-left quadrant displays the number of true negatives: the number of opinions that the model accurately classified as so.

The upper-right quadrant displays the number of false positives: the number of opinions that the model misclassified as claims.

The lower-left quadrant displays the number of false negatives: the number of claims that the model misclassified as opinions.

The lower-right quadrant displays the number of true positives: the number of claims that the model accurately classified as so.

A perfect model would yield all true negatives and true positives, and no false negatives or false positives.

As the above confusion matrix shows, this model produces five false negatives and five false positives.

Create a classification report that includes precision, recall, f1-score, and accuracy metrics to evaluate the performance of the model.

Note: In other labs there was a custom-written function to extract the accuracy, precision, recall, and F1 scores from the GridSearchCV report and display them in a table. You can also use scikit-learn's built-in `classification_report()` function to obtain a similar table of results.

```
# Create a classification report
# Create classification report for random forest model
### YOUR CODE HERE ###
target_labels = ['opinion', 'claim']
print(classification_report(y_val, y_pred,
target_names=target_labels))
```

	precision	recall	f1-score	support
opinion	1.00	1.00	1.00	1892
claim	1.00	1.00	1.00	1925
accuracy			1.00	3817
macro avg	1.00	1.00	1.00	3817
weighted avg	1.00	1.00	1.00	3817

Question: What does your classification report show? What does the confusion matrix indicate?

The classification report above shows that the random forest model scores were perfect, but we know they weren't quite perfect. The confusion matrix indicates that there were 10 misclassifications—five false positives and five false negatives.

XGBoost

Now, evaluate the XGBoost model on the validation set.

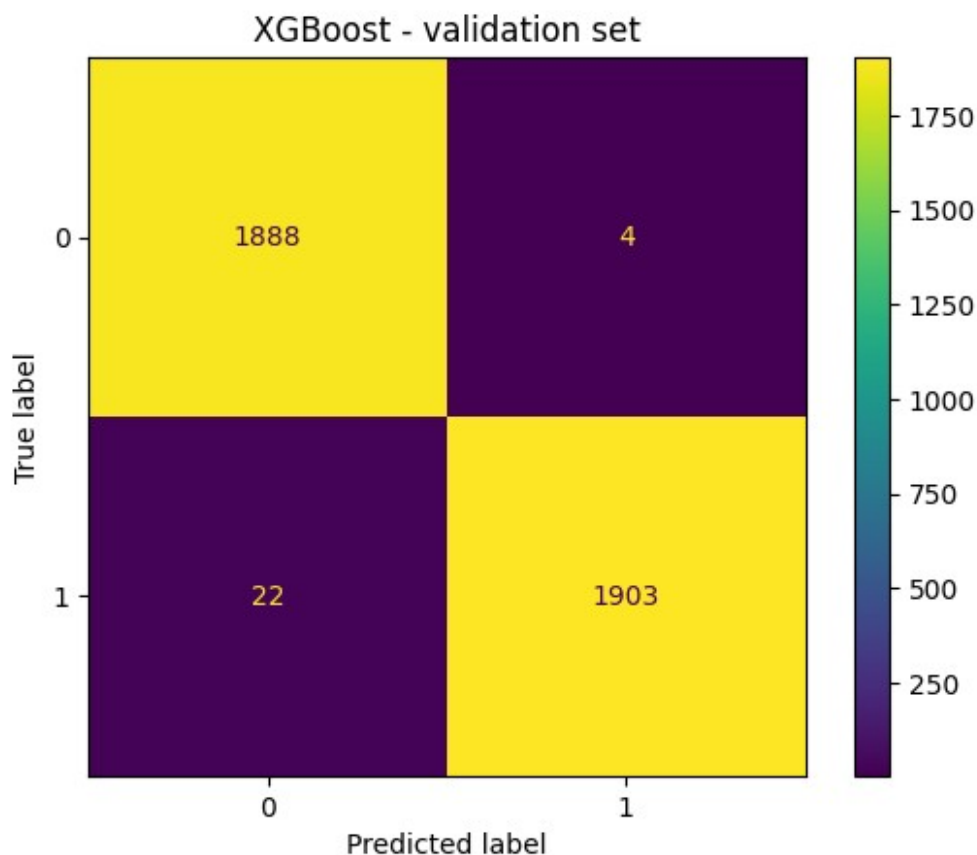
```
# Use the best estimator to predict on the validation data
### YOUR CODE HERE ###
y_pred = xgb_cv.best_estimator_.predict(X_val_final)

# Compute values for confusion matrix
log_cm = confusion_matrix(y_val, y_pred)

# Create display of confusion matrix
log_disp = ConfusionMatrixDisplay(confusion_matrix=log_cm,
display_labels=None)

# Plot confusion matrix
log_disp.plot()

# Display plot
plt.title('XGBoost - validation set');
plt.show()
```



```
# Create a classification report
### YOUR CODE HERE ###
target_labels = ['opinion', 'claim']
print(classification_report(y_val, y_pred,
target_names=target_labels))
```

	precision	recall	f1-score	support
opinion	0.99	1.00	0.99	1892
claim	1.00	0.99	0.99	1925
accuracy			0.99	3817
macro avg	0.99	0.99	0.99	3817
weighted avg	0.99	0.99	0.99	3817

Question: Describe your XGBoost model results. How does your XGBoost model compare to your random forest model?

Both random forest and XGBoost model architectures resulted in nearly perfect models. Nonetheless, in this case random forest performed a little bit better, so it is the champion model.

Now, use the champion model to predict on the test data.

Use champion model to predict on test data

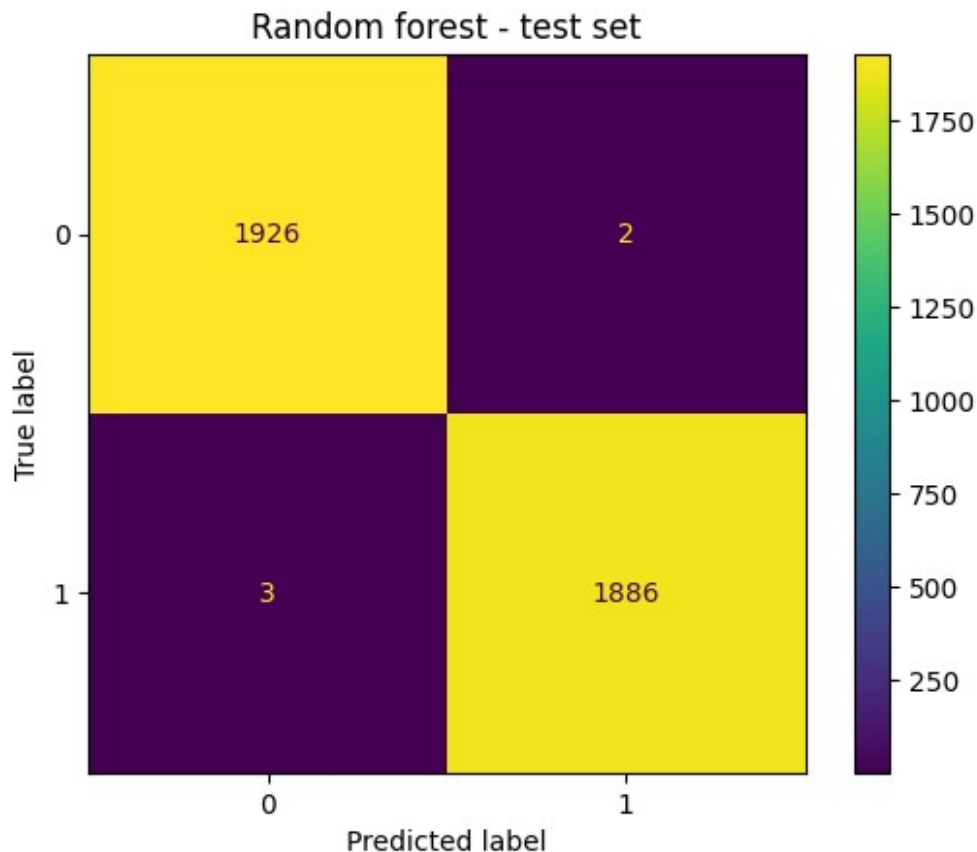
```
### YOUR CODE HERE ###
y_pred = rf_cv.best_estimator_.predict(X_test_final)

# Compute values for confusion matrix
log_cm = confusion_matrix(y_test, y_pred)

# Create display of confusion matrix
log_disp = ConfusionMatrixDisplay(confusion_matrix=log_cm,
display_labels=None)

# Plot confusion matrix
log_disp.plot()

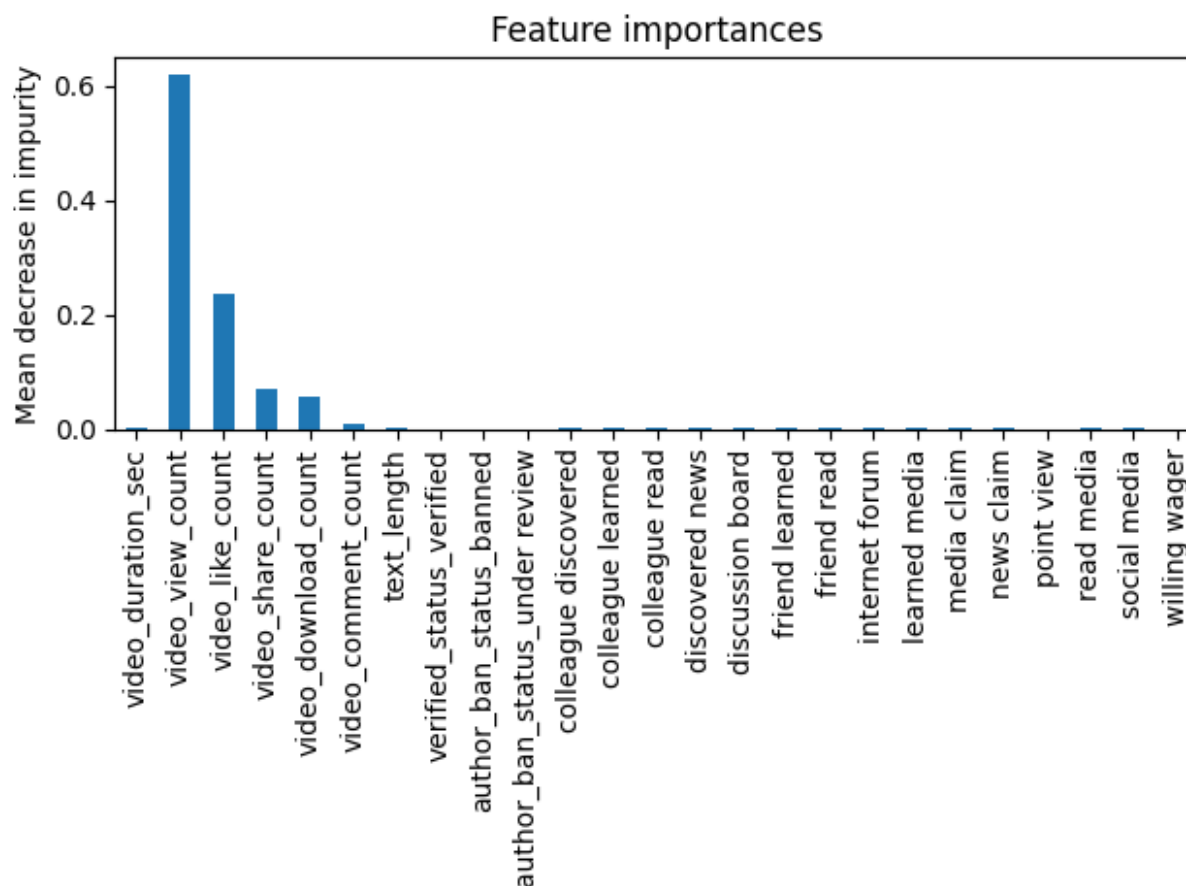
# Display plot
plt.title('Random forest - test set');
plt.show()
```



Feature importances of champion model

```
### YOUR CODE HERE ###
importances = rf_cv.best_estimator_.feature_importances_
rf_importances = pd.Series(importances, index=X_test_final.columns)
```

```
fig, ax = plt.subplots()
rf_importances.plot.bar(ax=ax)
ax.set_title('Feature importances')
ax.set_ylabel('Mean decrease in impurity')
fig.tight_layout()
```



Question: Describe your most predictive features. Were your results surprising?

The most predictive features all were related to engagement levels generated by the video. This is not unexpected, as analysis from prior EDA pointed to this conclusion.

Task 8. Conclusion

In this step use the results of the models above to formulate a conclusion. Consider the following questions:

1. **Would you recommend using this model? Why or why not?**
2. **What was your model doing? Can you explain how it was making predictions?**
3. **Are there new features that you can engineer that might improve model performance?**

4. **What features would you want to have that would likely improve the performance of your model?**

Remember, sometimes your data simply will not be predictive of your chosen target. This is common. Machine learning is a powerful tool, but it is not magic. If your data does not contain predictive signal, even the most complex algorithm will not be able to deliver consistent and accurate predictions. Do not be afraid to draw this conclusion.

1. *Would you recommend using this model? Why or why not?* Yes, one can recommend this model because it performed well on both the validation and test holdout data. Furthermore, both precision and F1 scores were consistently high. The model very successfully classified claims and opinions.
2. *What was your model doing? Can you explain how it was making predictions?* The model's most predictive features were all related to the user engagement levels associated with each video. It was classifying videos based on how many views, likes, shares, and downloads they received.
3. *Are there new features that you can engineer that might improve model performance?* Because the model currently performs nearly perfectly, there is no need to engineer any new features.
4. *What features would you want to have that would likely improve the performance of your model?* The current version of the model does not need any new features. However, it would be helpful to have the number of times the video was reported. It would also be useful to have the total number of user reports for all videos posted by each author.

Congratulations! You've completed this lab. However, you may not notice a green check mark next to this item on Coursera's platform. Please continue your progress regardless of the check mark. Just click on the "save" icon at the top of this notebook to ensure your work has been logged.